MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

Programação Dinâmica 1

- O que é
- Problemas clássicos
- Como reconstruir a resposta

Omena <jvrco>

Leonardo <las4>

Thawan <trs3>

CC BY NC SA

MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

O que é DP?

Backtracking

- Otimizações

+ Memoização

Backtracking

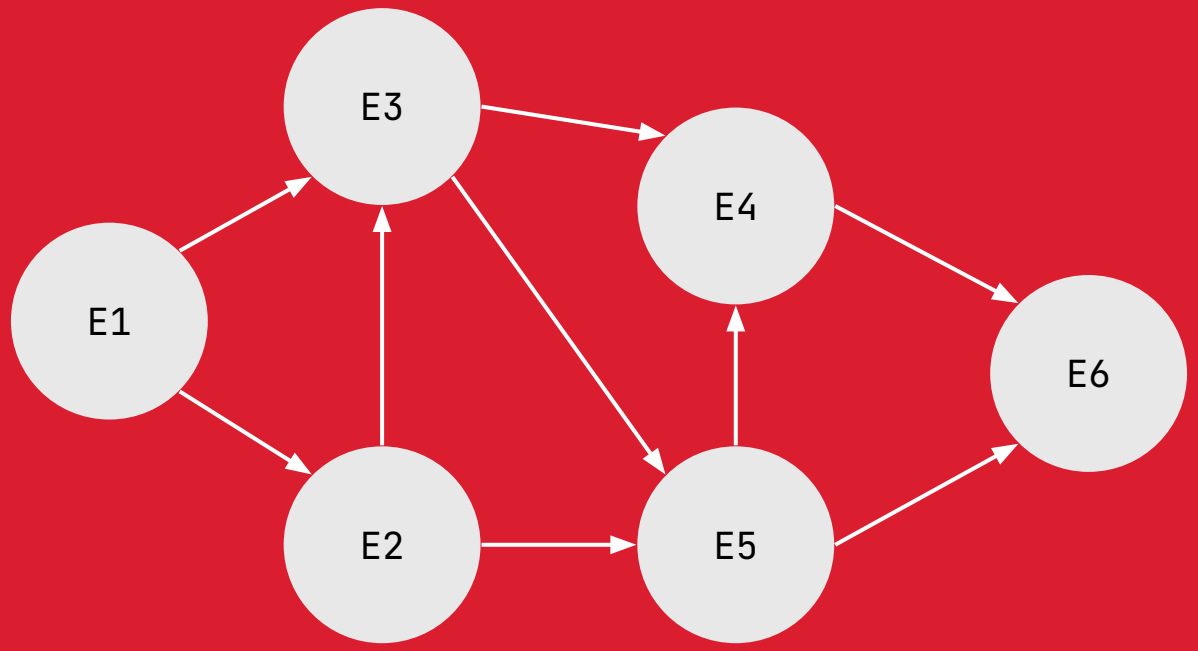
MaratonaCIn · Centro de Informática UFPE · Universidade Federal de Pernambuco

Estado

- Informação mínima para resolver o subproblema

Transição

- Escolha que te leva para outro estado



Problema

MaratonaCIn · Centro de Informática UFPE · Universidade Federal de Pernambuco

Dado um array A de tamanho N, encontre a maior soma possível de valores de uma subsequência de A em que cada par de elementos consecutivos tenha diferença absoluta menor ou igual a K.

- $N <= 10^5$
- $0 <= A[i], K <= 10$

Exemplos:

- `A = [8, 10, 6, 4, 2, 4]` | `K = 2 ⇒ [8, 6, 4, 2, 4] ⇒ 24`
- `A = [5, 2, 4, 3, 1],` | `K = 3 ⇒ [5, 2, 4, 3, 1] ⇒ 15`
- `A = [1, 4, 5, 6, 4, 3, 4, 2, 5]` | `K = 1 ⇒ [4, 5, 6, 5] ⇒ 20`

Solucionando...

- Vamos montar a subsequência ótima
- Cada elemento de A pode estar ou não estar na subsequência final
- Se o último elemento da subsequência atual tem valor X:
  - Se `|A[i] - X| ≤ K` e eu decido que A[i] vai estar na subsequência final, o próximo elemento, se houver, deve ser um A[j], tal que:
    - `j > i`
    - `|A[j] - A[i]| ≤ K`
  - Se eu decido que A[i] não vai estar na subsequência final, o próximo elemento, se houver, deve ser um A[j], tal que:
    - `j > i`
    - `|A[j] - X| ≤ K`

Solucionando...

MaratonaCIn Centro de Informática UFPE Universidade Federal de Pernambuco

- Vamos assumir que A[0] é o primeiro elemento da subsequência final
- Eu quero saber a **subsequência de A[1:] com maior soma e valores consecutivos com diferença menor ou igual a K** e **primeiro elemento com distância menor ou igual a K de A[0]**
- Se eu decido não colocar A[1] na subsequência:
  - Eu quero saber a **subsequência de A[2:] com maior soma e valores consecutivos com diferença menor ou igual a K** e **primeiro elemento com distância menor ou igual a K de A[0]**
- Se `|A[1] - A[0]| ⩽ K` e eu decido colocar A[1] na subsequência:
  - Eu quero saber a **subsequência de A[2:] com maior soma e valores consecutivos com diferença menor ou igual a K** e **primeiro elemento com distância menor ou igual a K de A[1]**

Solucionando...

MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

Eu quero saber a **subsequência de *<sufixo de A>*** **com maior soma e valores consecutivos com diferença menor ou igual a K** e **primeiro elemento com distância menor ou igual a K de *<elemento de A>***

Solucionando...

MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

Decidir se A[i] vai estar ou não na subsequência reduz o problema!

Backtracking

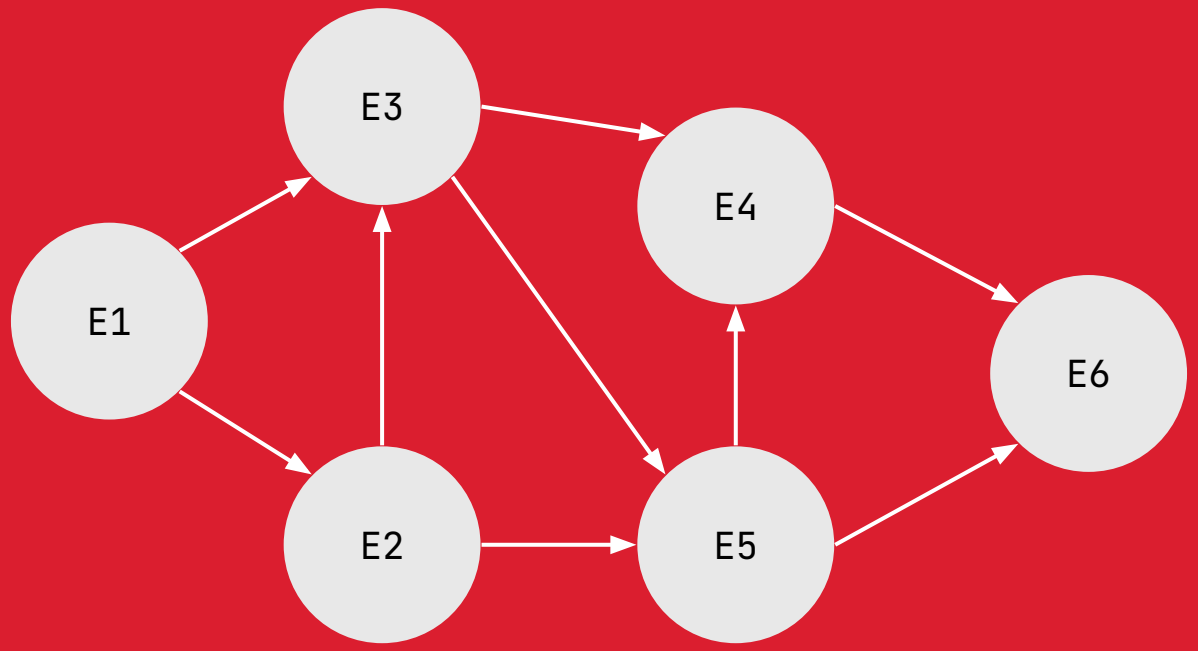
MaratonaCIn · Centro de Informática UFPE · Universidade Federal de Pernambuco

Estado

- Informação mínima para resolver o subproblema

Transição

- Escolha que te leva para outro estado



Backtracking

MaratonaCIn · Centro de Informática UFPE · Universidade Federal de Pernambuco

Estado

(i, X)

- i ⇒ Sufixo A[i:]
- X ⇒ último elemento da subsequência atual

Transição

Selecionar ou não A[i]

- (i, X) ⇒ (i + 1, X)
- (i, X) ⇒ (i + 1, A[i])

Backtracking

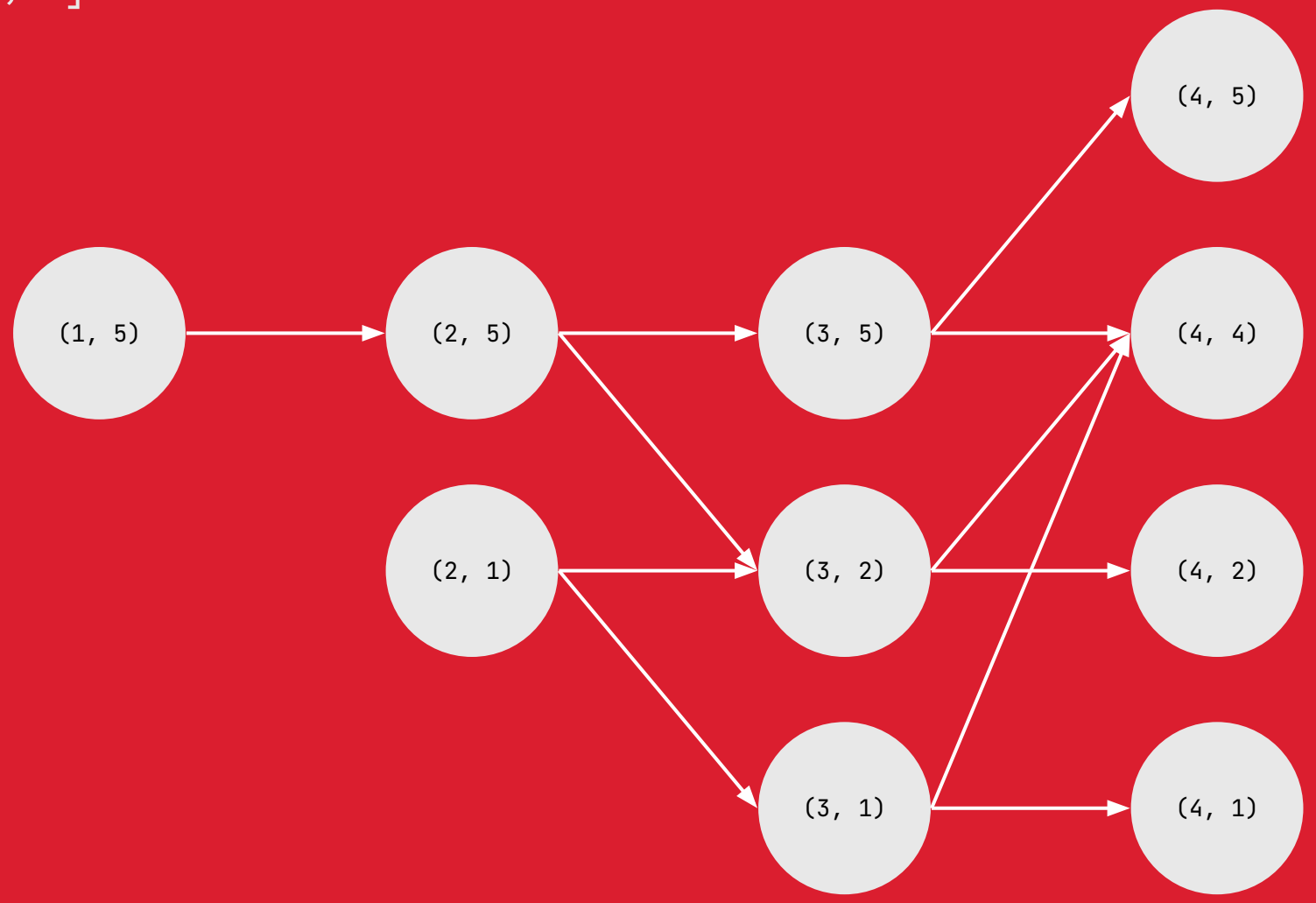
MaratonaCIn Centro de Informática UFPE Universidade Federal de Pernambuco

```
fun maior_soma(i, X):
    if i ⩾ N: return 0

    ans = maior_soma(i + 1, X)
    if |A[i] - X| ⩽ K:
        nans = maior_soma(i + 1, A[i]) + A[i]
        ans = max(ans, nans)

    return ans
```

$$\text{final_ans} = \max_{0 \leqslant i < N}(\text{maior_soma}(i + 1, A[i]) + A[i])$$

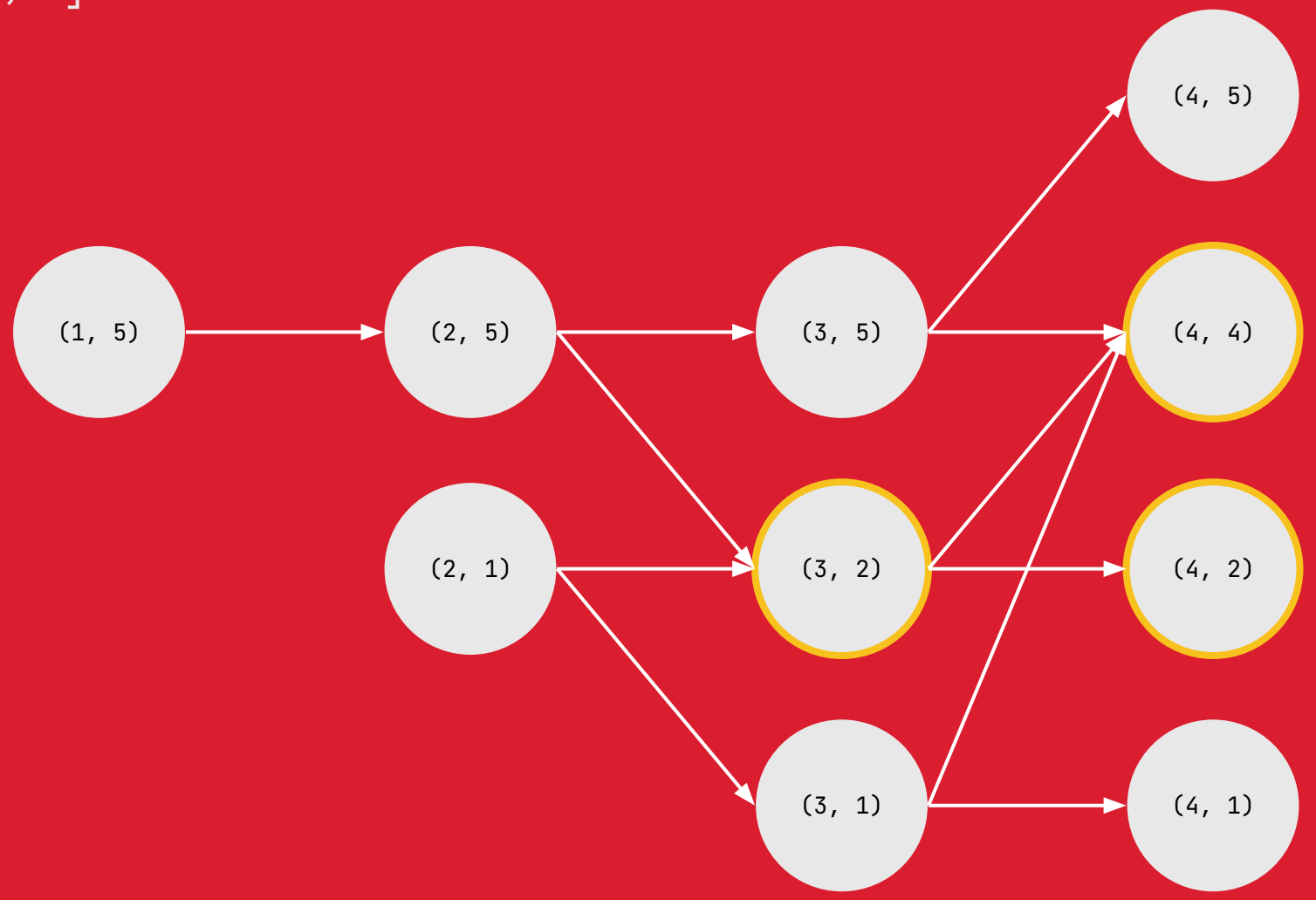
Backtracking

- A = [5, 1, 2, 4]
- K = 3



Backtracking

- A = [5, 1, 2, 4]
- K = 3



Backtracking

- Calculamos várias vezes a respostas para alguns estados
- O estado tem informação suficiente para resolver o subproblema, então a resposta é sempre a mesma, independente de como chegou lá

Backtracking

- Calculamos várias vezes a respostas para alguns estados
- O estado tem informação suficiente para resolver o subproblema, então a resposta é sempre a mesma, independente de como chegou lá

E se salvarmos os resultados em memória?

Memoização

MaratonaCIn · Centro de Informática UFPE · Universidade Federal de Pernambuco

```
fun maior_soma(i, X):
    if i ⩾ N: return 0
    if memo[i, X] ≠ null: return memo[i, X]

    memo[i, X] = maior_soma(i + 1, X)
    if |A[i] - X| ⩽ K:
        nans = maior_soma(i + 1, A[i]) + A[i]
        memo[i, X] = max(memo[i, X], nans)

    return memo[i, X]
```

$$\text{final_ans} = \max_{0 \leqslant i < N}(\text{maior_soma}(i + 1, A[i]) + A[i])$$

DP

- Computa todas as possibilidades de forma inteligente
- Salva as respostas de computações intermediárias
- Complexidade:
  - Memória: O(número de estados)
  - Tempo: O(número de estados * número de transições por estado)

MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

Problemas Clássicos

LIS

LIS

O que é o LIS (Longest Increasing Subsequence)?

LIS

MaratonaCIn Centro de Informática UFPE Universidade Federal de Pernambuco

O que é o LIS (Longest Increasing Subsequence)?

Dado um array, queremos achar o tamanho da maior subsequência estritamente crescente.

Importante: subsequência ≠ subarray (podemos pular elementos).

Exemplo:

Array = [10, 9, 2, 5, 3, 7, 101, 9, 20]

Pergunta: qual o tamanho do LIS?

LIS

MaratonaCIn Centro de Informática UFPE Universidade Federal de Pernambuco

O que é o LIS (Longest Increasing Subsequence)?

Dado um array, queremos achar o tamanho da maior subsequência estritamente crescente.

Importante: subsequência ≠ subarray (podemos pular elementos).

Exemplo:

Array = [10, 9, 2, 5, 3, 7, 101, 9, 20]

Pergunta: qual o tamanho do LIS?

LIS = [2, 3, 7, 9, 20]: tamanho = 5.

LIS

Método ingênuo:

Para cada número (maior que o anterior escolhido), você tem duas escolhas: usar ou não usar.

LIS

Método ingênuo:

Para cada número (maior que o anterior escolhido), você tem duas escolhas: usar ou não usar. $O(2^n)$

LIS

Método ingênuo:

Para cada número (maior que o anterior escolhido), você tem duas escolhas: usar ou não usar. $O(2^n)$

Com programação dinâmica:

dp[i] = tamanho da maior subsequência crescente que termina em i

LIS

Método ingênuo:

Para cada número (maior que o anterior escolhido), você tem duas escolhas: usar ou não usar. $O(2^n)$

Com programação dinâmica:

dp[i] = tamanho da maior subsequência crescente que termina em i $O(n^2)$

LIS

MaratonaCIn Centro de Informática UFPE Universidade Federal de Pernambuco

Com programação dinâmica:

dp[i] = tamanho da maior subsequência crescente que termina em i $O(n^2)$

```
int solve() {
    int n = 9;
    vector<int> a = {10, 9, 2, 5, 3, 7, 101, 9, 20};

    int answer = 0;
    vector<int> dp(n);
    for (int j = 0; j < n; ++j) {
        dp[j] = 1;
        for (int i = j - 1; i >= 0; --i) {
            if (a[i] < a[j]) {
                dp[j] = max(dp[j], dp[i] + 1);
            }
        }
        answer = max(answer, dp[j]);
    }
    return answer;
}
```

LIS

Método ingênuo:

Para cada número (maior que o anterior escolhido), você tem duas escolhas: usar ou não usar. $O(2^n)$

Com programação dinâmica:

dp[i] = tamanho da maior subsequência crescente que termina em i $O(n^2)$

Otimização: $O(n * \log n)$

LCS

LCS

O que é o LCS (Longest Common Subsequence)?

LCS

MaratonaCIn Centro de Informática UFPE Universidade Federal de Pernambuco

O que é o LCS (Longest Common Subsequence)?

LCS = maior subsequência comum entre duas strings, preservando a ordem, mas não necessariamente contígua.

Exemplo:

A = "acbd"

B = "abed"

Pergunta: qual o tamanho do LCS?

LCS

MaratonaCIn Centro de Informática UFPE Universidade Federal de Pernambuco

O que é o LCS (Longest Common Subsequence)?

LCS = maior subsequência comum entre duas strings, preservando a ordem, mas não necessariamente contígua.

Exemplo:

A = "acbd"

B = "abed"

Pergunta: qual o tamanho do LCS?

LIS = "abd": tamanho = 3.

LCS

Método ingênuo:

Gerar todas as subsequências de uma string e verificar quais também aparecem em ordem na outra.

LCS

Método ingênuo:

Gerar todas as subsequências de uma string e verificar quais também aparecem em ordem na outra. $O(2^n)$

LCS

Método ingênuo:

Gerar todas as subsequências de uma string e verificar quais também aparecem em ordem na outra. $O(2^n)$

Com programação dinâmica:

dp[i][j] = LCS entre A[0..i] e B[0..j]

LCS

Método ingênuo:

Gerar todas as subsequências de uma string e verificar quais também aparecem em ordem na outra. $O(2^n)$

Com programação dinâmica:

dp[i][j] = LCS entre A[0..i] e B[0..j]

Se A[i] == B[j]:

    dp[i][j] = dp[i - 1][j - 1] + 1

Senão:

    dp[i][j] = max(dp[i-1][j], dp[i][j-1])

LCS

Método ingênuo:

Gerar todas as subsequências de uma string e verificar quais também aparecem em ordem na outra. $O(2^n)$

Com programação dinâmica: $O(n * m)$

dp[i][j] = LCS entre A[0..i] e B[0..j]

Se A[i] == B[j]:

    dp[i][j] = dp[i - 1][j - 1] + 1

Senão:

    dp[i][j] = max(dp[i-1][j], dp[i][j-1])

LCS

```
vector<vector<int>> dp;

string a;
string b;

int best(int i, int j) {
    if (i < 0 or j < 0) return 0;
    if (dp[i][j] != -1) return dp[i][j];

    dp[i][j] = 0;
    if (a[i] == b[j]) {
        dp[i][j] = 1 + best(i - 1, j - 1);
    } else {
        dp[i][j] = max(best(i - 1, j), best(i, j - 1));
    }
    return dp[i][j];
}

int solve() {
    a = "acbd";
    int n = 4;
    b = "abed";
    int m = 4;

    dp = vector<vector<int>>(n, vector<int>(m, -1));

    return best(n - 1, m - 1);
}
```

entro de
formática
PE
Universidade
Federal
de Pernambuco

Subset Sum

Subset Sum

O que é o Subset Sum?

Subset Sum

O que é o Subset Sum?

É o problema de decidir se existe algum subconjunto de um conjunto de inteiros que some exatamente um valor-alvo T.

Subset Sum

MaratonaCIn Centro de Informática UFPE Universidade Federal de Pernambuco

O que é o Subset Sum?

É o problema de decidir se existe algum subconjunto de um conjunto de inteiros que some exatamente um valor-alvo T.

Exemplo simples:

Conjunto = {3, 4, 7, 2}

T = 9

Pergunta: existe subconjunto que soma 9?

Subset Sum

MaratonaCIn Centro de Informática UFPE Universidade Federal de Pernambuco

O que é o Subset Sum?

É o problema de decidir se existe algum subconjunto de um conjunto de inteiros que some exatamente um valor-alvo T.

Exemplo simples:

Conjunto = {3, 4, 7, 2}

T = 9

Pergunta: existe subconjunto que soma 9?

Sim: {7, 2} ou {3, 4, 2}

Subset Sum

Método ingênuo:

Para cada número, você tem duas escolhas: usar ou não usar.

E essas escolhas influenciam a soma atual.

Subset Sum

Método ingênuo:

Para cada número, você tem duas escolhas: usar ou não usar.

E essas escolhas influenciam a soma atual. $O(2^n)$

Subset Sum

Método ingênuo:

Para cada número, você tem duas escolhas: usar ou não usar.

E essas escolhas influenciam a soma atual. $O(2^n)$

Com programação dinâmica (2D):

dp[i][s] = é possível fazer soma s usando os primeiros i números?

Subset Sum

Método ingênuo:

Para cada número, você tem duas escolhas: usar ou não usar.

E essas escolhas influenciam a soma atual. $O(2^n)$

Com programação dinâmica (2D):

dp[i][s] = é possível fazer soma s usando os primeiros i números? $O(i * s)$

Subset Sum

Método ingênuo:

Para cada número, você tem duas escolhas: usar ou não usar.

E essas escolhas influenciam a soma atual. $O(2^n)$

Com programação dinâmica (2D):

dp[i][s] = é possível fazer soma s usando os primeiros i números? $O(i * s)$

Com programação dinâmica (1D):

dp[s] = consigo fazer soma s?

Subset Sum

Método ingênuo:

Para cada número, você tem duas escolhas: usar ou não usar.

E essas escolhas influenciam a soma atual. $O(2^n)$

Com programação dinâmica (2D):

dp[i][s] = é possível fazer soma s usando os primeiros i números? $O(i * s)$

Com programação dinâmica (1D):

dp[s] = consigo fazer soma s? $O(i * s)$

Subset Sum

MaratonaCIn · Centro de Informática UFPE · Universidade Federal de Pernambuco

Com programação dinâmica (1D):

dp[s] = consigo fazer soma s? O(i * s)

```
bool solve() {
    int t = 1;
    vector<int> a = {3, 4, 7, 2};

    vector<bool> dp(t + 1);
    dp[0] = true;

    for (int x : a) {
        for (int i = t - x; i >= 0; --i) {
            if (dp[i]) {
                dp[i + x] = true;
            }
        }
    }
    return dp[t];
}
```

MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

Knapsack

Knapsack

O que é o Knapsack?

Knapsack

MaratonaCIn Centro de Informática UFPE Universidade Federal de Pernambuco

O que é o Knapsack?

Você tem: uma mochila com capacidade W,

n itens, cada um com: peso w[i] e valor v[i]

Pergunta: qual o maior valor que dá pra colocar na mochila sem passar do peso?

Cada item pode ser pego 1 vez ou não pego

Knapsack

MaratonaCIn Centro de Informática UFPE Universidade Federal de Pernambuco

Exemplo:

Você tem: uma mochila com capacidade W = 7,

3 itens, cada um com: peso w[i] e valor v[i]

Item	Peso	Valor
1	3	4
2	4	5
3	2	3
4	6	7

Pergunta: qual o maior valor que dá pra colocar na mochila sem passar do peso?

Knapsack

Exemplo:

Você tem: uma mochila com capacidade W = 7,

3 itens, cada um com: peso w[i] e valor v[i]

Item	Peso	Valor
1	3	4
2	4	5
3	2	3
4	6	7

Pergunta: qual o maior valor que dá pra colocar na mochila sem passar do peso? 9

Knapsack

Método ingênuo:

O método ingênuo é simplesmente tentar todas as combinações possíveis de itens. Para cada item, você tem 2 escolhas: pegar (se couber) ou não pegar.

Knapsack

Método ingênuo:

O método ingênuo é simplesmente tentar todas as combinações possíveis de itens. Para cada item, você tem 2 escolhas: pegar (se couber) ou não pegar. $O(2^n)$

Com programação dinâmica (2D):

dp[i][w] = melhor valor usando itens até o i, com capacidade w

Knapsack

Método ingênuo:

O método ingênuo é simplesmente tentar todas as combinações possíveis de itens. Para cada item, você tem 2 escolhas: pegar (se couber) ou não pegar. $O(2^n)$

Com programação dinâmica (2D):

dp[i][w] = melhor valor usando itens até o i, com capacidade w $O(n * W)$

Com programação dinâmica (1D):

dp[w] = melhor valor com capacidade w

Knapsack

Método ingênuo:

O método ingênuo é simplesmente tentar todas as combinações possíveis de itens. Para cada item, você tem 2 escolhas: pegar (se couber) ou não pegar. $O(2^n)$

Com programação dinâmica (2D):

dp[i][w] = melhor valor usando itens até o i, com capacidade w $O(n * W)$

Com programação dinâmica (1D):

dp[w] = melhor valor com capacidade w $O(n * W)$

Knapsack

MaratonaCIn Centro de Informática UFPE Universidade Federal de Pernambuco

Com programação dinâmica (1D):

dp[w] = melhor valor com capacidade w $O(n * W)$

```
int solve() {
    int n = 4, W = 7;
    vector<int> w = {3, 4, 2, 6};
    vector<int> v = {4, 5, 3, 7};

    vector<int> knapsack(W + 1);
    for (int i = 0; i < n; ++i) {
        for (int k = W - w[i]; k >= 0; --k) {
            knapsack[k + w[i]] = max(knapsack[k + w[i]], knapsack[k] + v[i]);
        }
    }
    return knapsack[W];
}
```

MaratonaCIn Centro de Informática UFPE Universidade Federal de Pernambuco

como reconstruir a resposta

Como Reconstruir a Resposta

O que a DP normalmente faz:

Calcula o valor ótimo de um problema.

ex: menor custo, maior soma...

Mas como recuperamos as decisões que levaram a esse valor?

Muitas questões pedem qual caminho foi seguido,itens escolhidos e operações que foram feitas.

Como Reconstruir a Resposta

Caminho de maior soma em uma matriz
Problema: Sair de (1,1) até (3,3), somando o máximo possível e só se movimentando para baixo ou para direita

a[i][j]	j = 1	j = 2	j = 3
i = 1	5	1	4
i = 2	2	7	2
i = 3	3	1	9

dp[i][j]	j = 1	j = 2	j = 3
i = 1	5	6	10
i = 2	7	14	16
i = 3	10	15	**25**

dp[i][j] = a[i][j] + max(dp[i-1][j],dp[i][j-1]

Como Reconstruir a Resposta

dp[i][j]	j = 1	j = 2	j = 3
i = 1	**5**	6	10
i = 2	**7**	**14**	**16**
i = 3	10	15	**25**

rec[i][j]	j = 1	j = 2	j = 3
i = 1	start	(1,1)	(1,2)
i = 2	(1,1)	(2,1)	(2,2)
i = 3	(2,1)	(2,2)	**(2,3)**

Caminho encontrado: (1,1) → (2,1) → (2,2) → (2,3) → (3,3)

Como atualizar as tabelas

- dp[i] - Guarda o melhor valor para aquela posição
- rec[i] - Guarda de qual posição anterior veio o valor

```cpp
int n = 5;


  vector<long long> dp(n,LLONG_MIN);
  vector<int> rec(n, -1);

  int start = 0;
  dp[start] = 0;

  for (int u = 0; u < n; u++) {
    for (int v : adj[u]) {
      long long cand = dp[u] + w[u][v];

      if (cand > dp[v]) {
        dp[v] = cand;
        rec[v] = u;
      }
    }
  }

```

Como reconstruir a resposta

```
vector<int> caminho;

int s = estado_final;

while (s != estado_base) {
    caminho.push_back(s);
    s = rec[s];
}

caminho.push_back(estado_base);
reverse(caminho.begin(), caminho.end());
```

Dúvidas?

MaratonaCIn

Centro de Informática UFPE

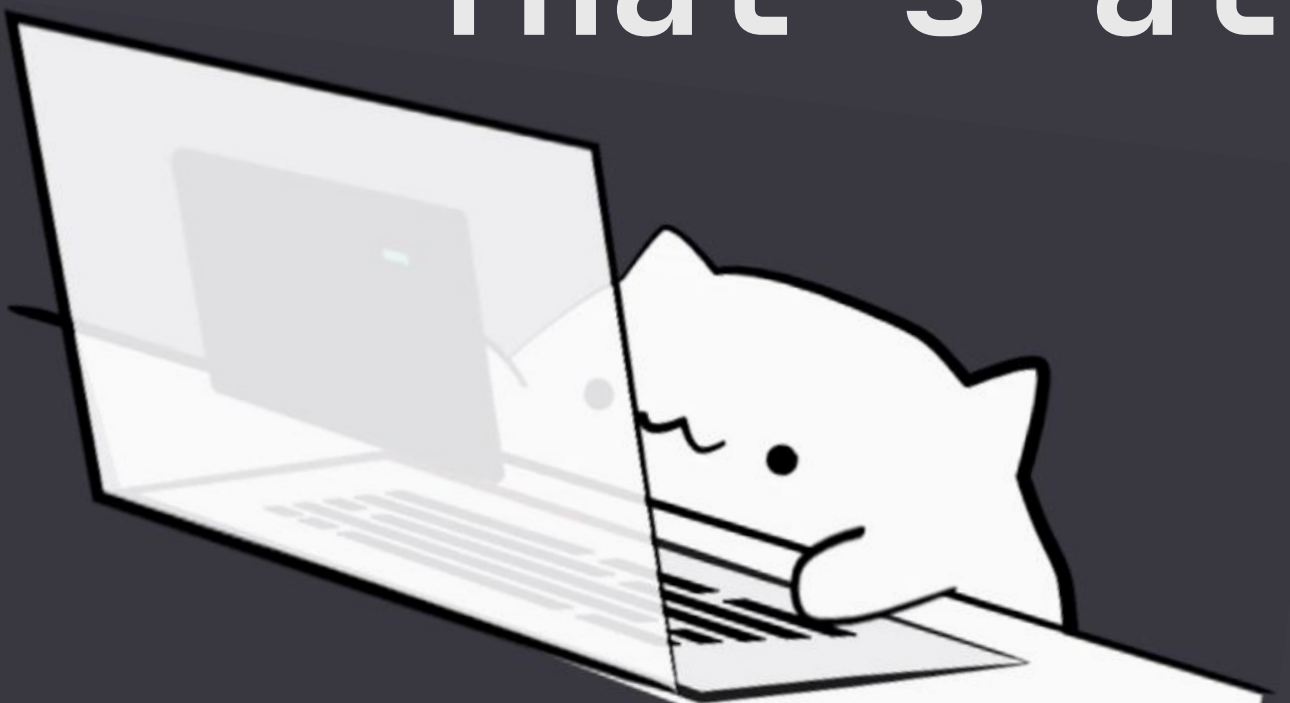
Universidade Federal de Pernambuco

Dúvidas?

Próximo passo:
Fazer o Homework!

MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

That’s all, Folks!